

Commandes Utiles pour gérer un fichier Docker-compose.yml

Commande	Explication
<code>docker compose up</code>	Démarre tous les services définis dans docker-compose.yml
<code>docker compose up --build</code>	Reconstruit les images + démarre les services (utile après modification du code)
<code>docker compose up -d</code>	Démarre les services en arrière-plan (mode détaché)
<code>docker compose down</code>	Arrête et supprime tous les containers + réseau du projet
<code>docker compose down -v</code>	Arrête + supprime les containers + supprime aussi les volumes (perte des données MongoDB)
<code>docker compose stop</code>	Arrête les containers sans les supprimer
<code>docker compose start</code>	Redémarre des containers déjà créés
<code>docker compose restart</code>	Redémarre tous les services
<code>docker compose ps</code>	Affiche l'état des containers du projet
<code>docker compose logs</code>	Affiche tous les logs des services
<code>docker compose logs -f</code>	Affiche les logs en temps réel (live)
<code>docker compose logs gateway</code>	Affiche les logs uniquement du service gateway
<code>docker compose exec gateway sh</code>	Ouvre un terminal dans le container gateway
<code>docker compose exec user-service sh</code>	Ouvre un shell dans user-service
<code>docker compose build</code>	Build uniquement les images sans lancer les containers
<code>docker compose pull</code>	Télécharge les images depuis Docker Hub (si utilisées)
<code>docker compose config</code>	Vérifie et affiche la configuration finale du fichier compose
<code>docker compose rm</code>	Supprime les containers arrêtés
<code>docker compose down --remove-orphans</code>	Supprime aussi les containers non définis dans le compose

Exercice corrigé :

Construire le fichier docker-compose associé à chaque commande

1. `docker run --name postgres -p 5432:5432 -e POSTGRES_PASSWORD=mysecretkey postgres`

```
postgres:
  image: postgres
  container_name: postgres
  ports:
    - "5432:5432"
  environment:
    POSTGRES_PASSWORD: mysecretkey
```

2. `docker build -t user-service-image ./user-service`
`docker run --name user-service -p 4000:4000 --link postgres user-service-image`

```
user-service:
  build: ./user-service
  container_name: user-service
  ports:
    - "4000:4000"
  depends_on:
    - postgres
```

3. `docker build -t product-service-image ./product-service`
`docker run --name product-service -p 4001:4001 --link postgres product-service-image`

```
product-service:
  build: ./product-service
  container_name: product-service
  ports:
    - "4001:4001"
  depends_on:
    - postgres
```

4. `docker build -t api-gateway-image ./api-gateway`
`docker run --name api-gateway -p 4002:4002 --link user-service --link product-service api-gateway-image`

```
api-gateway:
  build: ./api-gateway
  container_name: api-gateway
  ports:
    - "4002:4002"
  depends_on:
    - user-service
    - product-service
```